This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

PoPeC: PAoI-Centric Task Offloading With Priority Over Unreliable Channels

Nan Qiao, *Graduate Student Member, IEEE*, Sheng Yue, *Member, IEEE*, Yongmin Zhang, *Senior Member, IEEE*, and Ju Ren, *Senior Member, IEEE*

***Abstract*— Freshness-aware computation offloading has garnered increasing attention recently in the realm of edge computing, driven by the need to promptly obtain up-to-date information and mitigate the transmission of outdated data. However, most of the existing works assume that channels are reliable, neglecting the intrinsic fluctuations and uncertainty in wireless communication. More importantly, offloading tasks typically have diverse freshness requirements. Accommodation of various task priorities in the context of freshness-aware task scheduling and resource allocation remains an open and unresolved problem. To overcome these limitations, we cast the freshness-aware task offloading problem as a multi-priority optimization problem, considering the unreliability of wireless channels, prioritized users, and the heterogeneity of edge servers. Building upon the nonlinear fractional programming and the ADMM-Consensus method, we introduce a joint resource allocation and task offloading algorithm to solve the original problem iteratively. In addition, we devise a distributed asynchronous variant for the proposed algorithm to further enhance its communication efficiency. We rigorously analyze the performance and convergence of our approaches and conduct extensive simulations to corroborate their efficacy and superiority over the existing baselines.**

***Index Terms*— Distributed task offloading, edge computing, channel allocation, system freshness.**

I. Introduction

EDGE computing is an attractive computing paradigm in the era of the Artificial Internet of Things (AIoT) [1], [2]. By enabling end devices to offload computation-intensive tasks to nearby edge nodes, it is envisioned to provide real-time computing services, thereby facilitating the deployment of a wide range of intelligent applications (e.g., smart homes, smart cities, and autonomous vehicles) [3]. In these applications, it is of paramount importance to promptly access up-to-date information while mitigating the transmission of outdated and worthless data [4]. To this end, a great number of offloading solutions have been proposed to ensure timely status updates and rapid delivery of tasks from information sources, with the aim of enhancing the overall *information freshness* [5], [6], [7], [8], [9].

Recently, the Age of Information (AoI) and Peak Age of Information (PAoI) have been recognized as important metrics for evaluating the freshness of information, which characterize the elapsed time since the reception of a user's most recent data packet [10]. Based on these metrics, several recent efforts have focused on AoI- or PAoI-centric computation offloading and resource allocation for efficient and concurrent transmission of freshness-sensitive information to the edge servers [4], [5], [6], [7], [8], [9], [11], [12], [13], [14], [15], [16], [17].

Unfortunately, there remain several challenges that need to be surmounted to achieve effective freshness-aware computation offloading in practice. First, many existing works assume channel homogeneity [8], [18] or perfect knowledge of channel states [9], [19], overlooking the dynamics and stochasticity of the limited wireless channels. As a result, such methods easily suffer from package loss or failure due to unreliable communication [13]. Second, computing resources on edge servers are typically constrained and heterogeneous, necessitating appropriate assignment of heterogeneous computing units to offloading tasks. More importantly, designing a *prioritized offloading strategy* is crucial because users may have diverse freshness requirements. For example, devices with safety-sensitive functions, such as temperature sensors in the Industrial Internet of Things and Automatic Emergency Braking (AEB) systems in autopilots, require prompt offloading and processing to meet their stringent freshness demands. Whereas, most of the existing methods struggle with measuring and handling the situations where offloading tasks possess different priorities. In light of these considerations, this work seeks to answer the key question: *"How to design an efficient task offloading algorithm that can optimize the overall information freshness while effectively handling prioritized users, unreliable channels, and heterogeneous edge servers?"*

To this end, we cast the freshness-aware task offloading problem as a multi-priority optimization problem, considering unreliability of wireless channels, heterogeneity of edge servers, and interdependence of multiple users with differing

Manuscript received 25 September 2023; revised 15 December 2023; accepted 1 January 2024; approved by IEEE/ACM Transactions on Networking Editor V. Aggarwal. This work was supported in part by the National Natural Science Foundation of China under Grant 62341201, Grant 62122095, Grant 62072472, Grant U19A2067, Grant 92067206, Grant 62172445, and Grant 62302260; in part by the National Key Research and Development Program of China under Grant 2022YFF0604502; in part by the China Postdoctoral Science Foundation under Grant 2023M731956; and in part by the Grant from the Guoqiang Institute, Tsinghua University. *(Corresponding author: Ju Ren.)*

Nan Qiao and Yongmin Zhang are with the School of Computer and Engineering, Central South University, Changsha, Hunan 410083, China (e-mail: nan.qiao@csu.edu.cn; zhangyongmin@csu.edu.cn).

Sheng Yue is with the Department of Computer Science and Technology, BNRist, Tsinghua University, Beijing 100084, China (e-mail: shengyue@tsinghua.edu.cn).

Ju Ren is with the Department of Computer Science and Technology, BNRist, Tsinghua University, Beijing 100084, China, and also with the Zhongguancun Laboratory, Beijing 100094, China (e-mail: renju@tsinghua.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TNET.2024.3350198, provided by the authors.

Digital Object Identifier 10.1109/TNET.2024.3350198

1558-2566 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

priorities. Given the high complexity of directly optimizing this problem, we first examine two special cases from the original problem and exploit nonlinear fractional programming to transform the problems into tractable forms, subsequently developing ADMM-Consensus-based solutions for both cases. Built upon these solutions, an iterative algorithm is devised to resolve the original problem effectively. We further discuss a distributed asynchronous variant of the proposed algorithm, capable of alleviating the overhead caused by unreliable iterations during the offloading policy acquisition process. Theoretical analysis is carried out to establish the convergence property of the proposed algorithm and demonstrate the improvement in performance brought by the multi-priority mechanism.

In a nutshell, our main contributions are summarized below.

- We consider an M/G/1 offloading system and derive the precise Peak Age of Information (PAoI) expression for each user to characterize their information freshness. Then, we formulate the freshness-aware multi-priority task offloading problem under heterogeneous edge servers and unreliable channels.
- Based on nonlinear fractional programming and the ADMM-Consensus method, we propose a joint resource allocation, service migration, and task offloading algorithm to solve the original problem effectively. We further devise a distributed asynchronous variant for the proposed algorithm to enhance its communication efficiency.
- We establish theoretical guarantees for the proposed algorithms, in terms of performance and convergence. We conduct extensive simulations, and the results show that our algorithm can significantly improve the performance over the existing methods.

The remainder of the paper is organized as follows: Section II briefly reviews the related work. Section III introduces the system model, including relevant definitions and models. In Section IV, we describe some special cases and propose algorithms to tackle the PoPeC problem. Section V proposes an asynchronous parallel algorithm to improve communication efficiency and discusses the benefits of the multi-class priority mechanism. Section VI presents the simulation results, followed by a conclusion drawn in Section VII.

II. Related Work

In the realm of edge computing, a multitude of research efforts have emerged to mitigate response delays by means of task offloading strategies [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30]. On the one hand, exploring the characteristics of channels is integral to this field. The reliability of channels has been investigated in scenarios such as real-time monitoring systems [31], [32]. However, these approaches might not adequately address the challenges posed by unstable channel conditions stemming from factors like antenna beamforming and fading [33], [34]. Moreover, while earlier studies assumed either homogeneous channels or the offloading of two separate channels with random arrivals [8], [9], [18], such assumptions fall short when dealing with the complexities of heterogeneous unreliable channels. On the other hand, one notable departure in our work is the consideration of the performance of synchronous parallel iterative algorithms in the context of unreliable channels. While many studies advocate for distributed methods to enhance efficiency, they often neglect the substantial communication costs of synchronous parallel algorithms on unreliable channels [4], [5], [6]. In contrast, we explore the potential of an asynchronous parallel algorithm to mitigate communication overheads and achieve the same performance.

Nonetheless, the sole emphasis on delay reduction may not ensure the necessary freshness of information for users, as highlighted in the works of Kosta et al. [35] and Yates et al. [10]. This has led to the development of freshness-aware methodologies, leveraging metrics such as AoI and Peak Age of Information (PAoI) [4], [5], [6], [7], [8], [9], [11], [12], [36]. Therefore, recent advancements have delved into the customization of computation offloading strategies to accommodate distinct user types and preferences [4], [5], [6], [9], [37], [38], [39], [40], [41], [42], [43]. Zou et al. [4] introduce a novel partial-index approach that accurately characterizes indexing issues in heterogeneous multi-user multi-channel systems. Their SWIM framework optimizes resource allocation using maximum weights. Sun et al. [6] propose an age-aware scheduling strategy rooted in Lyapunov optimization to cater to diverse users, providing bounds on age that comply with throughput constraints. However, many of these contributions overlook the importance of user priorities, which is vital for practical prioritized systems [4], [5], [6], [9], [37].

In contrast to prior works, which may focus on specific aspects such as channel types, reliability, or algorithmic choices, our research amalgamates these elements to address the intricate interplay of AoI optimization, edge computing, and heterogeneous channels. By investigating the challenges unique to these intersections, we contribute to a more comprehensive understanding of real-time computation offloading in dynamic environments.

III. System Model

In this section, we introduce the system model, including the Mobile Edge Computing (MEC) architecture, the freshness model, and the problem formulation.

A. MEC Architecture

As shown in Fig.1, we consider a wireless network system consisting of a set of M mobile edge servers (denoted by $\mathcal{M}$). Each server $m \in \mathcal{M}$ serves a set of N_m users ($\mathcal{N}_m$), and each user $n \in \mathcal{N}$ can offload their tasks to the corresponding server m_n via a limited number of wireless channels (denoted by $\mathcal{C} \doteq \{1, \ldots, C\}$). Considering the effects of frequency-selective fading [4], we define $p_{n,c}$ as the probability of a successful transmission from user n through channel c to server m_n. In this context, users have (potentially) varying offloading priorities, represented as $\Delta \doteq \{1, 2, \ldots, \delta_{\max}\}$. Let $\mathcal{N}_m^\delta$ denote the set of users that are prioritized as $\delta \in \Delta$ and served by server m, which satisfies $\bigcup_{m\in\mathcal{M}} \bigcup_{\delta\in\Delta} \mathcal{N}_m^\delta = \bigcup_{m\in\mathcal{M}} \mathcal{N}_m = \mathcal{N}$. We represent $\mathcal{N}^\delta \doteq \bigcup_{m\in\mathcal{M}} \mathcal{N}_m^\delta$ as the set of users with priority δ across all servers. To simplify

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

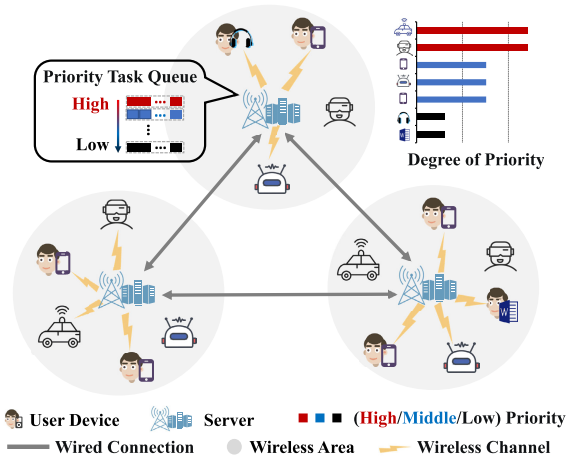


Fig. 1. MEC architecture with unreliable Channles and priority users.

TABLE I
KEY NOTATIONS

Notation	Definition
n, c, m, δ	indexes of users, channels, servers, and priority
u, s	superscripts of user-side and server-side variables
λ_n, μ_n	expected arrival and service rates of n's offloaded tasks
$p_{n,c}$	probability of a successful update from user n to channel c
y_m	optimal migration decision of server m
$\eta^u_{n,c}$	probability of tasks offloading from user n to channel c
$\eta^s_{m,m'}$	proportion of tasks delivered from server m to server m'
p^{syn}, p^{asyn}	probability of a successful (a)synchronous update
Γ^{syn}, Γ^{asyn}	number of iterations of (a)synchronous algorithms
T_n, I_n, W_n, Y_n	user n's transmission time, arrival interval, waiting time, and processing time
A_n	PAoI of user n's message
ϵ^{ck}, ϵ^{ac}	stop criteria for iterations CheckPointing Algorithm (NFP) and ADMM-Consensus Algorithm (NAC/ANAC)

notation, we denote the priority of user n as $\delta(n)$ and the set of users with the same or higher priority as $\Delta(\delta(n))$. Particularly, we distinguish user-side and server-side variables using the superscripts 'u' and 's', respectively. For clarity, we summarize the key notations in Table I.

1) Task Offloading: The computational tasks of each user arrive according to a Poisson process with an expected arrival rate of λ_n. We denote the probability of user n accessing channel c as $\eta^u_{n,c}$, which satisfies

$$0 \le \eta^u_{n,c} \le 1, \quad \forall n \in \mathcal{N},\ c \in \mathcal{C}. \tag{1}$$

Clearly, the number of tasks offloaded by user n through channel c also adheres to a Poisson progress with an expected value of $\eta^u_{n,c}\lambda_n$. Given the fact that the number of offloaded tasks by user n can not exceed that generated by the same user, we have

$$\sum_{c\in\mathcal{C}} \eta^u_{n,c} \le 1, \quad \forall n \in \mathcal{N}. \tag{2}$$

The inequality in Eq. (2) holds from the fact that users may discard outdated tasks due to congestion in the channel, or a need to prioritize more urgent tasks [44].

Edge servers are connected to each other via a wired network and can collaborate to execute offloaded tasks by assigning a portion of tasks from one server to another. We represent $\eta^s_{m,m'}$ as the proportion of tasks delivered from server m to server m', which satisfies

$$0 \le \eta^s_{m,m'} \le 1, \quad \forall m, m' \in \mathcal{M}, \tag{3}$$

$$\sum_{m'\in\mathcal{M}} \eta^s_{m,m'} = 1, \quad \forall m \in \mathcal{M}. \tag{4}$$

Accordingly, the number of computational tasks with priority δ delivered from server m to m' can be expressed as $\eta^s_{m,m'}\lambda^s_{\delta,m}$, where

$$\lambda^s_{\delta,m} \doteq \sum_{n\in\mathcal{N}^\delta_m} \sum_{c\in\mathcal{C}} p_{n,c}\lambda_{n,c}. \tag{5}$$

$\lambda^s_{\delta,m}$ denotes the total number of received tasks prioritized as δ at server m. Since the total number of tasks arrived at each server cannot exceed its maximum capacity (denoted by $\lambda^{s,\max}_m$), we have

$$\sum_{\delta\in\Delta} \sum_{m'\in\mathcal{M}} \eta^s_{m',m}\lambda^s_{\delta,m'} \le \lambda^{s,\max}_m, \quad \forall m \in \mathcal{M}. \tag{6}$$

2) Transmission Model: We assume that the transmission process of the multiple access channel follows the M/M/1 queuing model, as in [45] and [46].[1,2] We denote the task arrival rate of channel c as $\lambda_c \doteq \sum_{n\in\mathcal{N}} \eta^u_{n,c}\lambda_n$, *i.e.*, the sum of the offloading arrival rates of all users accessing this channel. Meanwhile, r_c stands for the communication for transmission rate through channel c, and S denotes the package size. As the currently achievable wireless communication rate approaches the Shannon limit [47], the transmission rate can be expressed as $r_c = B_c \log\big(R_c + 1\big)$, where B_c and R_c are the bandwidth and the signal-to-noise ratio of channel c, respectively. Thus, if user n accesses channel c, the expected transmission time can be expressed by

$$T^{\text{tr}}_{n,c} = \begin{cases} \dfrac{1}{r_c/S - \lambda_c} + t_{n,c}, & \dfrac{r_c}{S} - \lambda_c > 0, \\ \infty, & \text{otherwise}, \end{cases} \tag{7}$$

where $\frac{1}{r_c/S-\lambda_c}$ is the channel contention time, and $t_{n,c}$ is the constant end-to-end propagation delay of the offloading tasks.[3] The propagation delay, in a short time slice, is calculated by dividing the distance between the user and the server ($d_{n,c}$) by the speed at which the wireless signal propagates through the air (v_c), *i.e.*, $t_{n,c} = \frac{d_{n,c}}{v_c}$ [48]. As in [49], since MEC servers are linked via wired core networks, we assume that the end-to-end propagation delay between servers m and m' is constant, denoted by $t^{\text{tr}}_{m,m'}$.

B. Limited Capacity Model With Confidence Evaluation

To meet the requirements of real-time response, we consider the limited channel capacity/computation models with confidence, which enable real-time estimation and the control of system stability [5], [50].

[1]Task arrivals follow a Poisson process, and channel transmission times are exponential. It's like an M/M/1 queue, where tasks arrive randomly, wait in a queue, and are served one at a time, with exponential service times.

[2]The M/M/1 wireless transmission queue model can be simplified to the M/G/1 model when their second moments are equal, making them equivalent.

[3]Since $d_{n,c}$ does not change significantly relative to the speed of light v_c, $t_{n,c} = \frac{d_{n,c}}{v_c}$ can be regarded as a very small quantity or a constant.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

1) Channel Capacity: Denote the capacity of channel c as $M_c^{\max}$. Based on the properties of the cumulative distribution function of the Poisson distribution [51], we introduce the following constraint to ensure that each channel is conflict-free with confidence level $1-\alpha$ [5], [50]:

$$\frac{z_1^2}{2} + z_2 \cdot \left(\sum_{n\in\mathcal{N}} \eta_{n,c}^u \lambda_n + \frac{z_2^2}{4} \right)^{\frac{1}{2}} + \sum_{n\in\mathcal{N}} \eta_{n,c}^u \lambda_n \leq M_c^{\max}. \tag{8}$$

Here, $z_1 \doteq (\alpha/2 - \lambda_c^{\text{chl}})/\sqrt{\lambda_c^{\text{chl}}}$ and $z_2 \doteq (1-\alpha/2 - \lambda_c^{\text{chl}})/\sqrt{\lambda_c^{\text{chl}}}$ are two statistics standardized from a normal distribution, where $\lambda_c^{\text{chl}} \doteq \sum_{n\in\mathcal{N}} \eta_{n,c}^u \lambda_n$ represents the total number of tasks transmitted through channel c.

2) Computation Capacity: Due to the fluctuation in servers' available computing capacities, we assume that server m's processing time for user n's tasks follows a general distribution with mean $1/\mu_{n,m}$ and second moment $\nu_{n,m}$ [38].[4] To simplify notations, we use $1/\mu_n$ and ν_n to represent $1/\mu_{n,m_n}$ and ν_{n,m_n}, respectively.[5] To ensure each task is executed with a confidence level of $1-\beta$, we impose the following constraint:

$$\frac{z_3^2}{2} + z_4 \cdot \left(\sum_{n\in\mathcal{N}} \sum_{c\in\mathcal{C}} p_{n,c} \cdot \frac{\eta_{n,c}^u \lambda_n}{\mu_n} + \frac{z_4^2}{4} \right)^{\frac{1}{2}} + \sum_{n\in\mathcal{N}} \sum_{c\in\mathcal{C}} p_{n,c} \cdot \frac{\eta_{n,c}^u \lambda_n}{\mu_n} < 1, \tag{11}$$

where $z_3 \doteq (\beta/2 - \lambda_c^{\text{comp}})/\sqrt{\lambda_c^{\text{comp}}}$ and $z_4 \doteq (1-\beta/2 - \lambda_c^{\text{comp}})/\sqrt{\lambda_c^{\text{comp}}}$ are two statistics standardized from a normal distribution, with $\lambda^{\text{comp}} \doteq \sum_{n\in\mathcal{N}} \sum_{c\in\mathcal{C}} p_{n,c} \eta_{n,c}^u \lambda_n / \mu_n$ the total number of tasks arriving at server $m(n)$.

C. Freshness Model and Problem Formulation

We characterize the freshness of information using the PAoI metric. Specifically, the PAoI of user n's message, denoted by A_n, is determined by four key factors: transmission time T_n, arrival interval I_n, waiting time W_n, and processing time Y_n [4], *i.e.*,

$$\mathbb{E}[A_n] = \mathbb{E}[T_n] + \mathbb{E}[I_n] + \mathbb{E}[W_n] + \mathbb{E}[Y_n], \tag{12}$$

with $\mathbb{E}[I_n] = 1/\sum_{c\in\mathcal{C}} p_{n,c} \eta_{n,c}^u \lambda_n$ and $\mathbb{E}[T_n] = \sum_{c\in\mathcal{C}} \eta_{n,c}^u T_{n,c}^{\text{tr}}$. The expressions of $\mathbb{E}[W_n]$ and $\mathbb{E}[Y_n]$ are contingent upon the way in which user n's tasks are executed. To derive the precise expressions, we use binary variable y_m to denote the migration decision of server m, *i.e.*,

$$y_m \in \{0,1\}, \quad \forall m \in \mathcal{M}. \tag{13}$$

If y_m comes to 0, server m executes its tasks locally; otherwise, it resorts to the other servers for collaboration. In a slightly abusive notation, we use the superscript 'p' to represent the case when $y_m = 0$ and the superscript 's' to represent the case when $y_m = 1$. Thus, when $y_{m_n} = 0$, $\mathbb{E}[Y_n^p] = 1/\mu_n$, and $\mathbb{E}[W_n^p]$ can be expressed in Eq. (9), as shown at the bottom of the next page, from the M/G/1 queuing model with FCFS (see Appendix D-C of technical report [52] for more details). When $y_{m_n} = 1$, according to Little's Law,[6] we can obtain

$$\mathbb{E}[W_n^s] + \mathbb{E}[Y_n^s] = \sum_{m\in\mathcal{M}} \pi_{n,m}(\boldsymbol{\eta}^s), \tag{14}$$

where $\boldsymbol{\eta}^s \doteq \{\eta_{m,m'}^s\}_{m,m'\in\mathcal{M}}$, and $\pi_{n,m}(\boldsymbol{\eta}^s)$ is defined in Eq. (10), as shown at the bottom of the next page. Based on this, the expected PAoI of user n can be specified as follows:

$$\mathbb{E}[A_n] = (1-y_{m_n}) \cdot \mathbb{E}[A_n | y_{m_n} = 0] + y_{m_n} \cdot \mathbb{E}[A_n | y_{m_n} = 1]. \tag{15}$$

Given the aforementioned constraints, our objective is to find the optimal offloading decision for users and the collaboration decision for servers to minimize the average expected PAoI across all users. We cast the problem as the *PAoI-Centric Task Offloading with Priority over Unreliable Channels (PoPeC)*:

$$\textbf{(PoPeC)} \quad \min_{\boldsymbol{\eta}^u, \boldsymbol{\eta}^s, \boldsymbol{y}} \frac{1}{N} \sum_{n\in\mathcal{N}} \mathbb{E}[A_n], \quad \text{s.t. } (1)-(6), (8), (11), (13), \tag{16}$$

with $\boldsymbol{\eta}^u \doteq \{\eta_{n,c}^u\}_{n\in\mathcal{N}, c\in\mathcal{C}}$ the offloading decision, and $\boldsymbol{\eta}^s \doteq \{\eta_{m,m'}^s\}_{m,m'\in\mathcal{M}}$ along with $\boldsymbol{y} \doteq \{y_m\}_{m\in\mathcal{M}}$ the collaboration decision.

IV. PoPeC: PAoI-Centric Task Offloading With Priority Over Unreliable Channels

In light of the challenges in directly addressing the offloading problem, this section begins by examining two special cases – priority-free and multi-priority task scheduling with no server collaboration. Building on the insights gained from these solutions, we subsequently devise an effective and efficient algorithm for optimizing the original problem.

A. Priority-Free Task Scheduling

1) Problem Transformation: We first focus on a special case in which all tasks are of the same type and do not possess any priority distinctions, *i.e.*, the problem of *offloading to the local servers*. To solve this problem, the first step is to derive the expression of the expected PAoI for each user in this context. Recall that each user's task arrivals follow a Poisson distribution, and task execution times follow a general distribution. We re-evaluate the waiting time based on Eq. (9) as follows:

$$\mathbb{E}[W_n] = \frac{\sum_{n\in\mathcal{N}} \sum_{c\in\mathcal{C}} p_{n,c} \eta_{n,c}^u \lambda_n \nu_n}{2 \cdot \left(1 - \sum_{n\in\mathcal{N}} \sum_{c\in\mathcal{C}} p_{n,c} \cdot \frac{\eta_{n,c}^u \lambda_n}{\mu_n}\right)}. \tag{17}$$

[4]With task arrivals following a Poisson distribution, we consider an M/G/1 queueing system for server computations that adhere to an FCFS manner.

[5]$1/\mu_{n,m_n}$ and ν_{n,m_n} represent the mean and second moment, respectively, of the processing time for user n's tasks on its local server m_n.

[6]Little's law in queuing theory states that the average number of customers in a stationary system equals the product of arrival rate and waiting time [53].

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

We introduce an auxiliary variable, $\hat{t}_n^{\mathrm{tr}} > 0$, to represent the upper bound of user n's transmission time through all the available channels, *i.e.*

$$\hat{t}_n^{\mathrm{tr}} \geq T_{n,c}^{\mathrm{tr}}, \quad \forall c \in \mathcal{C}. \tag{18}$$

Akin to [54] and [55], we transform the original problem into the following tractable form, *i.e.*, optimizing a tight upper bound of user n's expected PAoI:

$$\textbf{(P1)} \quad \min_{\hat{t}_n^{\mathrm{tr}}, \boldsymbol{\eta}^u} \frac{1}{N} \sum_{n \in \mathcal{N}} f_n(\hat{t}_n^{\mathrm{tr}}, \boldsymbol{\eta}^u), \quad \text{s.t. } (1), (2), (8), (11), (18). \tag{19}$$

$f_n(\hat{t}_n^{\mathrm{tr}}, \boldsymbol{\eta}^u)$ is defined as:

$$f_n(\hat{t}_n^{\mathrm{tr}}, \boldsymbol{\eta}^u) \doteq \hat{t}^{\mathrm{tr}} + \frac{1}{\Psi_n(\boldsymbol{\eta}^u)} + \frac{\upsilon_n(\boldsymbol{\eta}^u)}{\phi_n(\boldsymbol{\eta}^u)} + \frac{1}{\mu}, \tag{20}$$

where we represent $\upsilon_n(\boldsymbol{\eta}^u) \doteq \frac{1}{2} \sum_{n' \in \mathcal{N}} \sum_{c \in \mathcal{C}} p_{n',c} \cdot \eta_{n',c}^u \boldsymbol{\lambda}_{n'} \nu$, $\phi_n(\boldsymbol{\eta}^u) \doteq 1 - \sum_{n' \in \mathcal{N}} \sum_{c \in \mathcal{C}} p_{n',c} \cdot \eta_{n',c}^u \boldsymbol{\lambda}_{n'} / \mu$, and $\Psi_n(\boldsymbol{\eta}^u) \doteq \sum_{c \in \mathcal{C}} p_{n,c} \eta_{n,c}^u \boldsymbol{\lambda}_n$.

In line with the principles of min-max optimization, we can enhance the overall communication efficiency across all channels by focusing our efforts on minimizing $\hat{t}_n^{\mathrm{tr}}$ rather than $\mathbb{E}[T_n]$. This strategic shift not only aligns with our objective of improving performance but also bolsters the resilience and robustness of our proposed solution.

Eq. (8) and Eq. (11) suggest that **P1** is a non-convex problem. To identify well-structured solutions, we transform these constraints into equivalent convex ones.

*Lemma 1: In Problem **P1**, Constraints (8) and (11) are equivalent to following Constraints (21) and (22) respectively*:

$$\sum_{n \in \mathcal{N}} \eta_{n,c}^u \boldsymbol{\lambda}_n \leq M_c^{\max} + \frac{z_2^2}{2} - \frac{z_1^2}{2} - z_2 \cdot \left(M_c^{\max} + \frac{z_2^2}{2} - \frac{z_1^2}{2} \right)^{\frac{1}{2}}, \tag{21}$$

$$\sum_{n \in \mathcal{N}} \sum_{c \in \mathcal{C}} p_{n,c} \frac{\eta_{n,c}^u \boldsymbol{\lambda}_n}{\mu_n} \leq 1 + \frac{z_4^2}{2} - \frac{z_3^2}{2} - z_4 \cdot \left(1 + \frac{z_4^2}{2} - \frac{z_3^2}{2} \right). \tag{22}$$

Proof: The detailed proof can be found in Appendix B of technical report [52]. ■

According to Lemma 1, Constraints (21) and (22) restrict the decision variables to convex sets, and hence, Problem **P1** can be rewritten as:

$$\textbf{(P1-1)} \quad \min_{\hat{t}_n^{\mathrm{tr}}, \boldsymbol{\eta}^u} \frac{1}{N} \sum_{n \in \mathcal{N}} f_n(\hat{t}_n^{\mathrm{tr}}, \boldsymbol{\eta}^u), \quad \text{s.t. } (1), (2), (18), (21), (22). \tag{23}$$

Building on the above transformation, we can obtain the following property.

*Theorem 1: Problem **P1-1** is a convex problem.*

Proof: It is easy to see that the constraints of **P1-1** are convex because they are affine sets. The convexity of all sub-functions is proven in Appendix C of technical report [52]. ■

Based on the convexity of Problem **P1-1**, we can employ the ADMM technique to solve this problem, detailed in the following section.

2) Problem Decomposition and Solving: We define an auxiliary function, $g_n(\boldsymbol{x})$, as follows:

$$g_n(\boldsymbol{x}) \doteq \begin{cases} \hat{t}^{\mathrm{tr}} + \frac{1}{\Psi_n(\boldsymbol{\eta}^u)} + \frac{\upsilon_n(\boldsymbol{\eta}^u)}{\phi_n(\boldsymbol{\eta}^u)} + \frac{1}{\mu}, & \boldsymbol{x} \in \Omega, \\ \infty, & \text{otherwise.} \end{cases} \tag{24}$$

where $\boldsymbol{x} \doteq \{\hat{t}^{\mathrm{tr}}, \boldsymbol{\eta}^u\}$, and $\Omega \doteq \{\boldsymbol{x} | (1), (2), (18), (21), (22)\}$ is the feasible set of Problem **P1-1**.

P1-1 is equivalent to the following consensus problem:

$$\textbf{(P1-2)} \quad \min_{\{\boldsymbol{x}_n\}} \sum_{n \in \mathcal{N}} g_n(\boldsymbol{x}_n), \tag{25}$$

$$\text{s.t. } \boldsymbol{x}_n = \boldsymbol{x}_o. \tag{26}$$

According to Theorem 1, it is clear that the well-known ADMM-Consensus algorithm can be used to obtain the optimal solution of Problem **P1-2** [56] (For more details, please refer to Appendix D of technical report [52]).

B. Multi-Priority Task Scheduling

1) Problem Transformation With Nonlinear Fractional Programming: Different from priority-free task scheduling in the previous section, we delve deeper into the multi-priority task scheduling problem in this section. In this case, according to the priorities of received tasks, servers will execute the tasks with higher priorities more promptly. Since the multi-priority task scheduling, in this case, is NP-hard (please refer to Appendix M of technical report [52] for the proof), similar to Problem **P1**, we minimize a tight upper bound for the

$$\mathbb{E}[W_n^p] = \frac{\frac{1}{2} \sum_{\delta \in \Delta} \sum_{n' \in \mathcal{N}^\delta} \sum_{c \in \mathcal{C}} p_{n',c} \eta_{n',c}^u \boldsymbol{\lambda}_{n'} \nu_{n'}}{\left(1 - \sum_{\delta \in \Delta(\delta(n))} \sum_{n' \in \mathcal{N}^\delta} \sum_{c \in \mathcal{C}} p_{n',c} \frac{\eta_{n',c}^u \boldsymbol{\lambda}_{n'}}{\mu_{n'}}\right) \left(1 - \sum_{\delta \in \Delta(\delta(n)-1)} \sum_{n' \in \mathcal{N}^\delta} \sum_{c \in \mathcal{C}} p_{n',c} \frac{\boldsymbol{\lambda}_{n',c}}{\mu_{n'}}\right)} \tag{9}$$

$$\pi_{n,m}(\boldsymbol{\eta}^s) \doteq \frac{\sum_{\delta \in \Delta} \eta_{m_n,m}^s \boldsymbol{\lambda}_{\delta,m_n}^s}{\sum_{\delta \in \Delta} \sum_{m' \in \mathcal{M}} \eta_{m_n,m'}^s \boldsymbol{\lambda}_{\delta,m_n}^s} \cdot \left(t_{m_n,m}^{\mathrm{tr}} + \frac{1}{\mu_{n,m}} + \frac{1}{1 - \sum_{\delta \in \Delta(\delta(n))} \sum_{m' \in \mathcal{M}} \eta_{m',m}^s \boldsymbol{\lambda}_{\delta,m'}^s \frac{1}{\mu_{n,m}}} \cdot \frac{\frac{1}{2} \sum_{\delta \in \Delta} \sum_{m' \in \mathcal{M}} \eta_{m',m}^s \boldsymbol{\lambda}_{\delta,m'}^s \nu_{n,m}}{1 - \sum_{\delta \in \Delta(\delta(n)-1)} \sum_{m' \in \mathcal{M}} \eta_{m',m}^s \boldsymbol{\lambda}_{\delta,m'}^s \frac{1}{\mu_{n,m}}} \right) \tag{10}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

average expected PAoI of multi-priority users:

$$\textbf{(P2)} \quad \min_{\boldsymbol{x}} \frac{1}{N} \sum_{n\in\mathcal{N}} f_n^p(\boldsymbol{x}), \quad \text{s.t. } (1), (2), (8), (11), (18), \tag{27}$$

where $f_n^p(\boldsymbol{x}) = \hat{t}^{\text{tr}} + \frac{1}{\mu_n} + \frac{1}{\Psi_n(\boldsymbol{x})} + \frac{\Upsilon(\boldsymbol{x})}{\Phi_{\delta(n)}(\boldsymbol{x})\Phi_{\delta(n)-1}(\boldsymbol{x})}$ is the upper bound of $\mathbb{E}[A_n]$, $\boldsymbol{x} = \{\hat{t}^{\text{tr}}, \boldsymbol{\eta}^u\}$ denotes the decision variables, $\Phi_{\delta(n)}(\boldsymbol{x}) = 1 - \sum_{\delta\in\Delta(\delta(n))}\sum_{n'\in\mathcal{N}^\delta}\sum_{c\in\mathcal{C}} p_{n',c}\eta^u_{n',c}\lambda_{n'}/\mu_{n'}$, $\Upsilon(\boldsymbol{x}) = \frac{1}{2}\sum_{\delta\in\Delta}\sum_{n'\in\mathcal{N}^\delta}\sum_{c\in\mathcal{C}} p_{n',c}\eta^u_{n',c}\lambda_{n'}\nu_{n'}$, and $\Psi_n(\boldsymbol{x}) = \sum_{c\in\mathcal{C}} p_{n,c}\eta^u_{n,c}\lambda_n$. To obtain the effective offloading decision in this case, we next transform the original problem and decouple users' offloading decisions.

First, we define $\theta_n \doteq f_n^{p,u}(\boldsymbol{x}_n)/f_n^{p,l}(\boldsymbol{x}_n) = f_n^p(\boldsymbol{x}_n) - 1/\mu_n$ and write

$$\min_{\boldsymbol{\theta}} \sum_{n\in\mathcal{N}} \theta_n = \min_{\boldsymbol{x}} \sum_{n\in\mathcal{N}} \frac{f_n^{p,u}(\boldsymbol{x}_n)}{f_n^{p,l}(\boldsymbol{x}_n)} = \sum_{n\in\mathcal{N}} \frac{f_n^{p,u}(\boldsymbol{x}_n^*)}{f_n^{p,l}(\boldsymbol{x}_n^*)}, \tag{28}$$

where $\boldsymbol{x}_n^*$ is the optimal solution and $f_n^{p,u}, f_n^{p,l}$ are denoted by

$$f_n^{p,u}(\boldsymbol{x}_n) = \Phi_{\delta(n)}(\boldsymbol{x}_n)\Phi_{\delta(n)-1}(\boldsymbol{x}_n) + \Upsilon(\boldsymbol{x}_n)\Psi_n(\boldsymbol{x}_n) + \hat{t}^{\text{tr}}\Psi_n(\boldsymbol{x}_n)\Phi_{\delta(n)}(\boldsymbol{x}_n)\Phi_{\delta(n)-1}(\boldsymbol{x}_n), \tag{29}$$

$$f_n^{p,l}(\boldsymbol{x}_n) = \Psi_n(\boldsymbol{x}_n)\Phi_{\delta(n)}(\boldsymbol{x}_n)\Phi_{\delta(n)-1}(\boldsymbol{x}_n). \tag{30}$$

*Proposition 1: Problem **P2** can be recast into an equivalent problem as follows*:

$$\textbf{(P2-1)} \quad \min_{\{\boldsymbol{x}_n\}} \sum_{n\in\mathcal{N}} f_n^{p,u}(\boldsymbol{x}_n) - \theta_n^* f_n^{p,l}(\boldsymbol{x}_n), \quad \text{s.t. } (1), (2), (18), (21), (22), \tag{31}$$

where θ_n^* is the minimum value of $f_n^{p,u}(\boldsymbol{x}_n)/f_n^{p,l}(\boldsymbol{x}_n)$ with regard to $\boldsymbol{x}_n$.

Proof: The detailed proof can be found in Appendix E of technical report [52]. ■

Proposition 1 demonstrates that problem transformation can be employed to deal with the nonlinear fractional programming problem **P2-1**. Hence, we can use the iterative Dinkelbach techniques [57], outlined in Algorithm 1, to solve the transformed problem. Algorithm 1 updates the value of $\tilde{\theta}_n$ in each iteration based on the current $\tilde{\boldsymbol{x}}_n^k$, *i.e.*,

$$\tilde{\theta}_n^{k+1} = \frac{f_n^{p,u}(\tilde{\boldsymbol{x}}_n^{k+1})}{f_n^{p,l}(\tilde{\boldsymbol{x}}_n^{k+1}))}. \tag{32}$$

This process continues until:

$$f_n^{p,u}(\tilde{\boldsymbol{x}}_n^{k+1}) - \tilde{\theta}_n^k f_n^{p,l}(\tilde{\boldsymbol{x}}_n^{k+1})) > \epsilon^{\text{ck}}, \tag{33}$$

where ϵ^{ck} represents the stop criteria for iterations.

2) ADMM-Consensus Based Solution: The procedure mentioned above can be considered as a checkpointing algorithm. We employ non-convex ADMM-Consensus methods to determine the new value of $\{\boldsymbol{x}_n\}$. Analogous to section IV-A, the consensus problem for **P2-1** can be expressed as:

$$\textbf{(P2-2)} \quad \min_{\{\boldsymbol{x}_n\}} \sum_{n\in\mathcal{N}} g_n^p(\boldsymbol{x}_n), \quad \text{s.t. } \boldsymbol{x}_n = \boldsymbol{x}_o, \tag{34}$$

Algorithm 1 Nonlinear Fractional Programming Based on ADMM-Consensus (NFPA)

```
// CheckPointing Algorithm - NFP
Input: ε^ck, {θ̃_n^0}, k
for n = 1 to N do
    Convergence = False
    while Convergence = False do
        Apply Algorithm 2 or 4 with x̃_n^k and θ̃_n^k to
        obtain the solution x̃_n^{k+1}
        if f_n^{p,u}(x̃_n^{k+1}) − θ̃_n^k f_n^{p,l}(x̃_n^{k+1})) > ε^ck then
            Select θ̃_n^{k+1} = f_n^{p,u}(x̃_n^{k+1}) / f_n^{p,l}(x̃_n^{k+1}))
            Convergence = True
        else
            Convergence = False
        end
        Update s = s + 1
    end
    Select x_n^{p,*} = x̃_n^k
    Select θ_n^{p,*} = θ̃_n^k
    Calculate PAoI according to Eq. (27)
    Output: x_n^{p,*} and PAoI
end
```

where

$$g_n^p(\boldsymbol{x}_n) = \begin{cases} f_n^{p,u}(\boldsymbol{x}_n) - \theta_n^* f_n^{p,l}(\boldsymbol{x}_n), & \boldsymbol{x} \in \Omega, \\ \infty, & \text{otherwise}. \end{cases} \tag{35}$$

The augmented Lagrangian for Problem **P2-2** can be expressed as:

$$\begin{aligned} L^p(\{\boldsymbol{x}_n\}, \boldsymbol{x}_o, \{\boldsymbol{\sigma}_n\}) &= \sum_{n\in\mathcal{N}} L_n^p(\boldsymbol{x}_n, \boldsymbol{x}_o, \boldsymbol{\sigma}_n) \\ &= \sum_{n\in\mathcal{N}} \left(g_n^p(\boldsymbol{x}_n) + \langle \boldsymbol{\sigma}_n, \boldsymbol{x}_n - \boldsymbol{x}_o \rangle + \frac{\rho_n}{2}\|\boldsymbol{x}_n - \boldsymbol{x}_o\|_2^2 \right), \end{aligned} \tag{36}$$

where ρ_n is a positive penalty parameter with respect to Problem **P2-2**. Based on the non-convex ADMM-Consensus algorithm, we update the variables in each iteration t as follows:

$$\boldsymbol{x}_n^{t+1} = \arg\min_{\boldsymbol{x}_n} L_n^p(\{\boldsymbol{x}_n\}, \boldsymbol{x}_o^t, \{\boldsymbol{\sigma}_n^t\}), \tag{37}$$

$$\boldsymbol{x}_o^{t+1} = \frac{\sum_{n\in\mathcal{N}}(\rho_n \boldsymbol{x}_n^{t+1} + \sigma_n^t)}{\sum_{n\in\mathcal{N}} \rho_n}, \tag{38}$$

$$\sigma_n^{t+1} = \sigma_n^t + \rho_n(\boldsymbol{x}_n^{t+1} - \boldsymbol{x}_o^{t+1}). \tag{39}$$

The premise that the iterative update can converge is that the function g_n^p is Lipschitz continuous.

Proposition 2: The first-order derivative of g_n^p is Lipschitz continuous with constant ℓ_n, which is defined as

$$\ell_n = \frac{\lambda_{max}^2}{\mu_{\min}^2}\left(\sum_{n_1\in\mathcal{N}} \frac{\nu_n}{2\mu_n\mu_{n_1}} + (1 + \theta_n^* \frac{\nu_n}{2\mu_n})|\mathcal{N}_1| + \theta_n^* \frac{\nu_n}{2\mu_n}|\mathcal{N}_2| \right), \tag{40}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

where $\mu_{\min} = \min_n\{\mu_n\}$, $\boldsymbol{\lambda}_{max} = \max_n \boldsymbol{\lambda}_n$, $\mathcal{N}_1 = \{\mathcal{N}^\delta | \delta < \delta(n)\} \bigcup \{n\}$, $\mathcal{N}_2 = \{\mathcal{N}^\delta | \delta \leq \delta(n)\}$.

Proof: The detailed proof can be found in Appendix F of technical report [52]. ■

According to Proposition 2, we adopt the non-convex ADMM-Consensus method to solve Problem **P2-2**, with given $\left(\{\boldsymbol{x}_n^0\}, \{\theta_n^*\}\right)$ (as outlined in Algorithm 1).

Algorithm 2 Non-Convex ADMM-Consensus (NAC)

Input: ϵ^{ac}, t and $\left(\{\boldsymbol{x}_n^0\}, \{\theta_n^*\}\right)$ from Algorithm 1

```
// ADMM-Consensus Algorithm
do
    Calculate x_o^{t+1} in MEC, according to Eq. (38)
    Calculate x_n^{t+1} in user, simultaneously according to Eq. (37)
    Calculate σ_n^{t+1} in user, simultaneously according to Eq. (39)
    Update t = t + 1
while η(x^t, σ^t) ≥ ε^ac;
```

Output: $\boldsymbol{x}_n^t$

It is worth mentioning that the value of ρ_n and the convergence property of the Algorithm 2 differ from those in the Convex ADMM-Consensus method. Due to the non-convexity of the objective function, the commonly used *gap function* cannot be adapted to the analysis of Algorithm 2. Next, we design a special gap function capable of characterizing the convergence of NAC as follows:

$$\eta\left(\boldsymbol{x}^t, \boldsymbol{\sigma}^t\right) = \left\|\tilde{\nabla} L^p\left(\{\boldsymbol{x}_n^t\}, \boldsymbol{x}_o^t, \boldsymbol{\sigma}^t\right)\right\|^2 + \sum_{n\in\mathcal{N}} \left\|\boldsymbol{x}_n^t - \boldsymbol{x}_o^t\right\|^2, \tag{41}$$

where

$$\tilde{\nabla} L^p\left(\{\boldsymbol{x}_n\}, \boldsymbol{x}_o, \boldsymbol{\sigma}^t\right) = \begin{bmatrix} \nabla_{\boldsymbol{x}_o} L^p\left(\{\boldsymbol{x}_n\}, \boldsymbol{x}_o, \boldsymbol{\sigma}^t\right) \\ \nabla_{\boldsymbol{x}_1} L^p\left(\{\boldsymbol{x}_n\}, \boldsymbol{x}_o, \boldsymbol{\sigma}^t\right) \\ \vdots \\ \nabla_{\boldsymbol{x}_N} L^p\left(\{\boldsymbol{x}_n\}, \boldsymbol{x}_o, \boldsymbol{\sigma}^t\right) \end{bmatrix}.$$

When $\eta\left(\boldsymbol{x}^t, \boldsymbol{\sigma}^t\right) < \epsilon^{\text{ac}}$, Algorithm 2 will find a stationary solution. The gap function that ADMM used to deal with convex functions is no longer suitable to judge the convergence of non-convex functions ADMM. We further explain why the gap function Eq. (41) can be used as a stopping criterion (A comprehensive explanation of this concept can be found in Appendix L of technical report [52].) In addition to the change of the gap function, the convergence of Algorithm 2 requires some parameters to satisfy special conditions.

Theorem 2: If $\rho_n > 2\ell_n$, *Algorithm 2 converges to an* ϵ^{ac}*-stationary point within* $O(1/(p^{\text{syn}}\epsilon^{\text{ac}}))$, *where* ϵ^{ac} *is a positive iteration factor, and* $p^{\text{syn}} = \Pi_{n\in\mathcal{N}}(\frac{1}{C}\sum_{c\in\mathcal{C}} p_{n,c})$ *is the probability of successfully completing a synchronous update.*

Proof: The detailed proof can be found in Appendix G of technical report [52]. ■

Theorem 2 implies that when specific parameters satisfy certain criteria, Algorithm 2 can sublinearly converge. Additionally, since the iteration will continue if the parameters given by Algorithm 2 do not satisfy the stop condition of Algorithm 1, we can use Algorithm 1 as a condition for the termination of the entire solution method.

C. Multi-Priority Task Scheduling and Multi-Server Collaboration

1) Problem Decomposition: Different from Section IV-A and Section IV-B, which concentrate solely on a single server, we further study multi-priority and multi-server collaboration-based offloading in this subsection. In order to resolve the original problem, we first derive the expression of the optimal migration decision variable $\boldsymbol{y}^*$.

*Theorem 3: The optimal migration decision of (**PoPeC**) is*

$$y_{m_n} = \begin{cases} 1, & \phi^{in}(\boldsymbol{\eta}^s) + \phi^{out}(\boldsymbol{\eta}^s) > 0, \\ 0, & otherwise, \end{cases} \tag{42}$$

where $\phi^{in}(\boldsymbol{\eta}^s) = \sum_{m'\in\mathcal{M}/m_n} \eta^s_{m',m_n} \sum_{\delta\in\Delta} \boldsymbol{\lambda}^s_{\delta,m'}$ *and* $\phi^{out}(\boldsymbol{\eta}^s) = \sum_{m'\in\mathcal{M}/m_n} \eta^s_{m_n,m'} \sum_{\delta\in\Delta} \boldsymbol{\lambda}^s_{\delta,m_n}$.

Proof: The detailed proof can be found in Appendix H-A of technical report [52]. ■

Combining Theorem 3 and Appendix H-B of technical report [52], **(PoPeC)** is equivalent to the following problem:

$$\textbf{(P3)} \quad \min_{\boldsymbol{x},\boldsymbol{z}} \frac{1}{N} \sum_{n\in\mathcal{N}} F_n^1(\boldsymbol{x}, \boldsymbol{z}) \quad \text{s.t. } (1), (2), (3), (4), (6), (18), (21), (22), \tag{43}$$

where $F_n^1(\boldsymbol{x}, \boldsymbol{z}) = \hat{t}_n^{\text{tr}} + \frac{1}{\sum_{c\in\mathcal{C}} p_{n,c}\eta^u_{n,c}\lambda_n} + \sum_{m\in\mathcal{M}} \pi_{n,m}(\boldsymbol{z})$, $\boldsymbol{x} = \{\hat{t}^{\text{tr}}, \boldsymbol{\eta}^u\}$ and $\boldsymbol{z} = \boldsymbol{\eta}^s$. The main challenge in solving **P3** lies in the non-convexity of $\sum_{n\in\mathcal{N}} F_n^1(\boldsymbol{x}, \boldsymbol{z})$ and the dependency of $\boldsymbol{x}$ and $\boldsymbol{z}$. We further decompose the problem **P3**.

Lemma 2: Problem **P3** can be equivalently transformed into the **Channel Allocation** subproblem, **P3-1**, and the **Server Collaboration** subproblem, **P3-2**, which are shown as follow:

$$\textbf{(P3-1)} \quad \min_{\boldsymbol{x}} \frac{1}{N} \sum_{n\in\mathcal{N}} F_n^2(\boldsymbol{x}), \quad \text{s.t. } (1), (2), (18), (21), (22), \quad \sum_{n\in\mathcal{N}_m^\delta} \sum_{c\in C} p_{n,c}\eta^u_{n,c}\lambda_n \leq \boldsymbol{\lambda}^s_{\delta,m}. \tag{44}$$

$$\textbf{(P3-2)} \quad \min_{\boldsymbol{z}} \frac{1}{N} \sum_{n\in\mathcal{N}} F_n^3(\boldsymbol{z}, \boldsymbol{\lambda}^s), \quad \text{s.t. } (3), (4), (6), \tag{45}$$

where we define $F_n^2(\boldsymbol{x}) \doteq \hat{t}_n^{\text{tr}} + \frac{1}{\sum_{c\in\mathcal{C}} p_{n,c}\eta^u_{n,c}\lambda_n}$, $F_n^3(\boldsymbol{z}, \boldsymbol{\lambda}^s) \doteq \sum_{m\in\mathcal{M}} \pi_{n,m}(\boldsymbol{z})$, and $\boldsymbol{\lambda}^s_{\delta,m} \doteq \sum_{m'\in\mathcal{M}} \eta^s_{m,m'} \boldsymbol{\lambda}^s_{\delta,m}$.

Proof: The detailed proof can be found in Appendix I of technical report [52]. ■

Based on the lemma provided, we develop methods to address the channel allocation **P3-1** and server cooperation **P3-2** iteratively on the user and server sides, respectively.

2) Channel Allocation: The goal of **P3-1** is to allocate channels for each local server and the user it serves. Based on Lemma 2, we can easily obtain the optimal solution of this sub-problem as follows:

$$\boldsymbol{x}^* = \{\boldsymbol{x}_m^*\}, \tag{46}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

where $\boldsymbol{x}_m^*$ can be derived from

$$\textbf{(P3-3)} \ \min_{\boldsymbol{x}_m} \sum_{n\in\mathcal{N}_m} F_n^2(\boldsymbol{x}_m, \boldsymbol{\lambda}^s), \quad \text{s.t. } (1), (2), (18), (21), (22), . \tag{47}$$

We further analyze **P3-3** to identify an optimal solution.

*Lemma 3: Problem **P3-3** is a convex problem.*

Proof: We can easily obtain the convexity of F_n^1 from Appendix C of technical report [52]. Since both the sub-function and the constraints are convex, we yield the result. ■

Based on Lemma 3 and the solution of IV-A, **P3-3** can be resolved by the existing method AC in Algorithm 5. The details can be found in Section D and Appendix D of technical report [52]. Each local server can find an optimal solution $\boldsymbol{x}_m^*$ for all users it covers, which is in fact the optimal channel allocation for **P3-1**.

3) Server Collaboration: Problem **P3-2** seeks to address the problem of multi-server collaboration between various servers in order to lessen the load on overhead servers and speed up task execution to decrease PAoI for multi-priority users, which has been shown to be an NP-Hard problem in section IV-B. However, we develop an effective migration strategy, based on Lemma 3, the server collaboration strategy is $\boldsymbol{z}^* = \arg\min_{\boldsymbol{z}}\{\frac{1}{N}\sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}}\pi_{n,m}(\boldsymbol{z})$ s.t. (3), (4), (6)$\}$.

*Proposition 3: Given $\boldsymbol{\lambda}^s$, Problem **P3-2** is equivalent to the following problem*:

$$\textbf{(P3-4)} \ \min_{\{\boldsymbol{z}_{n,m}\}} \sum_{n\in\mathcal{N}}\sum_{m\in\mathcal{M}} \{\pi_{n,m}^u(\boldsymbol{z}_{n,m}) - \vartheta_{n,m}^*\pi_{n,m}^l(\boldsymbol{z}_{n,m})\}, \quad s.t. \ (1), (2), (8), (11), (18). \tag{48}$$

Given $\boldsymbol{z}_{n,m}$, $\vartheta_{n,m}^*$ is the minimum of $\frac{\pi_{n,m}^u(\boldsymbol{z}_{n,m})}{\pi_{n,m}^l(\boldsymbol{z}_{n,m})}$, where

$$\begin{cases} \pi_{n,m}^l(\boldsymbol{z}) = \Phi_{\delta(n),m}^\pi(\boldsymbol{z})\Phi_{\delta(n)-1,m}^\pi(\boldsymbol{z}) \sum_{m'\in\mathcal{M}} \Psi_{n,m'}^\pi(\boldsymbol{z}), \\ \pi_{n,m}^u(\boldsymbol{z}) = \Lambda_{n,m}\Psi_{n,m}^\pi(\boldsymbol{z})\Phi_{\delta(n),m}^\pi(\boldsymbol{z})\Phi_{\delta(n)-1,m}^\pi(\boldsymbol{z}) \\ \qquad + \Psi_{n,m}^\pi(\boldsymbol{z})\Upsilon^\pi(\boldsymbol{z}), \\ \Phi_{\delta(n),m}^\pi(\boldsymbol{z}) = 1 - \sum_{\delta\in\Delta(\delta(n))}\sum_{m'\in\mathcal{M}} \eta_{m',m}^s \boldsymbol{\lambda}_{\delta,m'}^s \frac{1}{\mu_{n,m}}, \\ \Upsilon_{n,m}^\pi(\boldsymbol{z}) = \frac{1}{2}\sum_{\delta\in\Delta}\sum_{m'\in\mathcal{M}} \eta_{m',m}^s \boldsymbol{\lambda}_{\delta,m'}^s \nu_{n,m}, \\ \Psi_{n,m}^\pi(\boldsymbol{z}) = \sum_{\delta\in\Delta} \eta_{m_n,m}^s \boldsymbol{\lambda}_{\delta,m_n}^s, \\ \Lambda_{n,m} = t_{m_n,m}^{\text{tr}} + \frac{1}{\mu_{n,m}}. \end{cases}$$

Proof: $\pi_{n,m}^u(\boldsymbol{z})$ and $\pi_{n,m}^l(\boldsymbol{z})$ are polynomial functions representing the numerator and denominator of the fraction $\pi_{n,m}(\boldsymbol{z})$, respectively. Accordingly, we can reframe problem **P3-2** using nonlinear fractional programming, resulting in an equivalent problem denoted as **P3-4** with given $\boldsymbol{\lambda}^s$. For more detailed problem transformation, please refer to Appendix J-1 of technical report [52]. ■

Combining Proposition 3 and Section IV-B.1, we can apply NFP in Algorithm 1 to convert **P3-2** to **P3-4**. After transforming the problem in Proposition 3, we obtain problem **P3-4**, which has a cubic polynomial objective function. Nevertheless, deriving the closed-form solution for **P3-4** is still challenging, we instead provide an iterative algorithm as follows. We first analyze the properties of **P3-4**.

*Lemma 4: In **P3-4**, the first-order derivative of $\pi_{n,m}^u(\boldsymbol{z}_{n,m}) - \vartheta_{n,m}^*\pi_{n,m}^l(\boldsymbol{z}_{n,m})$ is Lipschitz continuous.*

Proof: The detailed proof can be found in Appendix J-2 of technical report [52]. ■

Based on Lemma 4 and Section IV-B.2, we use NAC and NFP in Algorithm 2 to gain an efficient solution of **P3-4**. In this case, the complexity of the method is $O(1/\epsilon^{\text{ac}})$, which can be proved by Appendix J-3 of technical report [52].

4) Iterative Solution: We design an iterative solution algorithm that first obtains the initial $\boldsymbol{x}_o$ inside each local server with a given $\boldsymbol{y}$. In the algorithm, $\boldsymbol{x}^t$ and $\boldsymbol{z}^t$ are solved alternately to obtain the solution. In the following, we establish the convergence guarantee for the proposed algorithm.

Algorithm 3 Iterative Solution (IS)

Input: $t = 0$, $\boldsymbol{\lambda}_{\delta,m}^{s,0} = \boldsymbol{\lambda}_m^{s,\max}$
while *not done* **do**
  Compute $\boldsymbol{x}^{t+1}$ according to Eq. (46) by Lemma 3
  Compute $\boldsymbol{\lambda}^{s,t+1}$ according to Eq. (5)
  Compute $\boldsymbol{z}^{t+1}$ according to Eq. (48) by Lemma 4
  Update $t = t + 1$
end
Compute $\boldsymbol{y}^*$ by Eq. (42)
Output: $\boldsymbol{x}^* = \boldsymbol{x}^t$,$\boldsymbol{y}^*$,$\boldsymbol{z}^* = \boldsymbol{z}^t$

*Theorem 4: If we solve **P3** by Algorithm 3, $F_n(\boldsymbol{x}^t, \boldsymbol{z}^t)$ monotonically decreases and converges to a unique point.*

Proof: The detailed proof can be found in Appendix K of technical report [52]. ■

V. Discussion

In this section, we expand upon and analyze the performance of PoPeC. Firstly, we introduce a communication-efficient asynchronous parallel algorithm and investigate both its convergence and convergence rate. Following that, we delve into the distinctions and benefits of our approach when compared to non-priority and traditional multi-priority methods.

A. Asynchronous Parallel Algorithm

In the previous sections, we propose several synchronous parallel algorithms. Nevertheless, the reliability and effectiveness of these algorithms can be severely affected by communication failures and chaos resulting from faulty communication networks. As Theorem 2 has revealed, Algorithm 2 has a slow convergence speed, highlighting the need to replace it with a more communication-efficient alternative.

Firstly, users opt to prioritize the selection of the channel with the highest reliability rate to maximize the success rate during iterations, represented as $p_n^{\max} = \max_c\{p_{n,c}\}$. In doing so, the risk of communication iteration loss is intuitively reduced. To ensure successful iterations in an asynchronous algorithm, there needs to be a limit on the number of communications required for each communication unit. In order

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Algorithm 4 Asynchronous Non-Convex ADMM-Consensus (ANAC)

/* MEC Side */
Input: ϵ^{ac}, τ_o and $\left(\{\boldsymbol{x}_n^0\}, \{\theta_n^*\}\right)$ from Algorithm 1
Convergence = False
while *Convergence == False* **do**
  Calculate $\boldsymbol{x}_o^{\tau_o+1} = \arg\min_{\boldsymbol{x}_o \in \Omega} L^p\left(\{\boldsymbol{x}_n^0\}, \boldsymbol{x}_o, \{\boldsymbol{\sigma}_n^0\}\right)$ **if** $\eta\left(\boldsymbol{x}^{\tau_o}, \boldsymbol{\sigma}^{\tau_o}\right) < \epsilon^{\text{ac}}$ **then**
    Convergence = True
    Send $\boldsymbol{x}_o^{\tau_o+1}$ and Convergence to all **users**
  else
    Send $\boldsymbol{x}_o^{\tau_o+1}$ and Convergence to all **users**
    Wait for some fixed period of time
    Receive all the gradients $\{x_n^{\tau_n^r}\}$ and all the local time $\{\boldsymbol{\tau}_n^r\}$ from **users**
    Record the received users in $\mho^{\tau_o+1}$
    if $n \in \mho^{\tau_o+1}$ **then**
      Select $\tau_n = \min\{\boldsymbol{\tau}_n^r\}$ and $\nabla G_n^{p,\tau_o+1} = x_n^{\tau_n^r}$
    else
      Select $\nabla G_n^{p,\tau_o+1} = \nabla G_n^{p,\tau_o}$
    end
    Calculate $\boldsymbol{x}_n^{\tau_o+1} = \boldsymbol{x}_o^{\tau_o+1} - \frac{1}{\rho_n}(\nabla G_n^{p,\tau_o+1} + \boldsymbol{\sigma}_n^{\tau_o})$
    Calculate $\boldsymbol{\sigma}_n^{\tau_o+1} = \boldsymbol{\sigma}_n^{\tau_o} + \rho_n(\boldsymbol{x}_n^{\tau_o+1} + \boldsymbol{x}_o^{\tau_o+1})$
    Update $\tau_o = \tau_o + 1$
  end
end
Output: $\boldsymbol{x}_o^{\tau_o}$
/* USER Side */
for *user* $n \in \mathcal{N}$ **do**
  Initialize $\tau_n = 0$
  while *Receive* $\boldsymbol{x}^r$ *and Convergence from* ***MEC*** **do**
    if *Convergence == False* **then**
      Select $\boldsymbol{x}_n^{\tau_n+1} = \boldsymbol{x}^r$
      Calculate $\nabla g_n^p(\boldsymbol{x}_n^{\tau_n+1})$
      Send $\nabla g_n^p(\boldsymbol{x}_n^{\tau_n+1})$ and (τ_n+1) to **MEC** via the most reliable channel
      Update $\tau_n = \tau_n + 1$
    else
      Select $\boldsymbol{x}_n = \boldsymbol{x}^r$
      Output: $\boldsymbol{x}_n$
    end
  end
end

to satisfy the delay bound of iterations needed for successful communication Γ_n, we have $(1-p_n^{\max})^{\Gamma_n} < \epsilon^a$, where ϵ^a is the maximum tolerance for asynchronous iterative communication. Both sides are logarithmic at the same time, it is $\Gamma_n \geq \frac{ln(\epsilon^a)}{ln(1-p_n^{\max})}$. Thus, we have

Assumption 1: The upper bound on the number of communications to complete a successful iteration satisfies

$$\Gamma_n = \left\lceil \frac{ln(\epsilon^a)}{ln(1-p_n^{\max})} \right\rceil, \tag{49}$$

where $\lceil x \rceil$ is the ceiling function of x.

This is a standard assumption in the asynchronous ADMM literature [58], [59]. In the worst case, it is a necessary condition to ensure that each user finishes one iteration within Γ_n iterations.

Furthermore, we present an asynchronous variant of Algorithm 2 as Algorithm 4. In each iteration, each user computes the gradient based on the most recently received information from the server and sends it to the local server. The server collects all available iterations, updates the value of $\boldsymbol{x}$, and passes the latest information back to the user. This asynchronous approach can help reduce communication overhead and improve convergence speed. In addition, we recognize that users may have limited computational resources, and thus, we suggest assigning a minimal number of computational tasks or designing the tasks to be as simple as possible.

Convergence Analysis: If Assumption 1 is satisfied and we set $\rho_n > \max\{7\ell_n, \ell_n(\Gamma_n^2 + \frac{3}{7}(\Gamma_n+1)^2)\}$, the sequence $\{\{\boldsymbol{x}_n\}, \boldsymbol{x}\}$ in Algorithm 4 converges to the set of stationary solutions of the problem, based on [59, Theorem 3.1]. Moreover, for $\epsilon^{\text{ac}} > 0$, we obtain

$$p^{\text{asyn}}\Gamma^{\text{asyn}}\epsilon^{\text{ac}} < k^{\Gamma}(L^p\left(\{\boldsymbol{x}_n^1\}, \boldsymbol{x}_o^1, \boldsymbol{\sigma}^1\right) - \underline{G^p}), \tag{50}$$

where ϵ^{ac} is a positive iteration factor, k^{Γ} is a constant, $\underline{G^p}$ is the lower bound of $\sum_{n\in\mathcal{N}} g_n^p(\boldsymbol{x}_n)$, Γ^{asyn} is number of iterations, *i.e.*, $\Gamma^{\text{asyn}} = \min\{t \mid \eta(\boldsymbol{x}^t, \boldsymbol{\sigma}^t) \leq \epsilon, t \geq 0\}$, $p^{\text{asyn}} = 1 - \Pi_{n\in\mathcal{N}}(1-p_n^{\max})$ denote the probability that at least one communication unit communicates successfully in an iteration and $p^{\text{asyn}}\Gamma^{\text{asyn}}$ represents the number of successful iterations. According to detailed analysis and proof in Appendix L of technical report [52], we have

$$\Gamma^{\text{asyn}} < \frac{k^{\Gamma}(L^p\left(\{\boldsymbol{x}_n^1\}, \boldsymbol{x}_o^1, \boldsymbol{\sigma}^1\right) - \underline{G^p})}{\epsilon^{\text{ac}}p^{\text{asyn}}}, \tag{51}$$

which means Algorithm 4 converges to an ϵ^{ac}-stationary point within $O(1/(p^{\text{asyn}}\epsilon^{\text{ac}}))$. Therefore, the asynchronous parallel algorithm is approximately $p^{\text{asyn}}/p^{\text{syn}}$ times faster, in comparison to the synchronous parallel approach according to Theorem 2. The value of $p^{\text{asyn}}/p^{\text{syn}}$ is greater than 1, and it increases as the channel quality declines.

Such convergence analysis shows that Algorithm 4 converges faster than Algorithm 2, particularly when transmission reliability is low. Furthermore, we demonstrate the effectiveness of the asynchronous algorithm through numerous simulation experiments. In each iteration, we are required to compute $\boldsymbol{x}o^{\tau o+1} = \arg\min_{\boldsymbol{x}o\in\Omega} \mathcal{L}^p\left(\boldsymbol{x}_n^0, \boldsymbol{x}o, \boldsymbol{\sigma}_n^0\right)$, where $\mathcal{L}^p$ is a convex problem. We can exploit widely-used gradient descent or interior point methods to solve this problem with low computational cost. In particular, we introduce an asynchronous parallel communication algorithm tailored for the issue of unreliable channels in priority-free cases as well (see Appendix D-C of technical report [52] for more details).

B. Why Multi-Class Priority?

Existing multi-priority methods focus on scenarios where each user has a unique priority level, which can lead to the unjust treatment of users who should have the same priority, as exemplified in previous studies such as [38], [41], and [43].

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

This may undermine the principles of fairness and equality that these systems are intended to uphold. However, the multi-priority model can extend to handle different scenarios to promote fair allocation, including cases where users have unequal priorities or where multiple users share the same priority level. In our research, this model is called multi-class priority, which aims to prevent unfair treatment of users with similar priorities by allocating resources equitably. To the best of our knowledge, our study represents the first attempt to explore such a priority model with a broader range of practical applications.

Next, we explore the benefits of our proposed multi-class priority mechanism in contrast to the commonly used multi-priority and no-priority mechanisms. Specifically, we delve into the effects of our approach on the performance of three distinct user groups:

a) The highest priority users: The PAoI of user $n*$, who are set to the highest priority ($\mathbb{E}[A^p_{n^*}]$), has more optimal information freshness than the priority-free case ($\mathbb{E}[A_{n^*}]$) under the same offloading strategy, which is

$$\begin{aligned}&\mathbb{E}[A_{n^*}] - \mathbb{E}[A^p_{n^*}]\\&= \Upsilon(\boldsymbol{\eta}^u)\frac{\zeta_\Delta(\boldsymbol{\eta}^u) - \zeta_{\delta(n^*)}(\boldsymbol{\eta}^u)}{(1-\zeta_\Delta(\boldsymbol{\eta}^u))(1-\zeta_{\delta(n^*)}(\boldsymbol{\eta}^u))}\\&\geq 0.\end{aligned} \tag{52}$$

where $\zeta_{\delta(n^*)}(\boldsymbol{\eta}^u) = \sum_{\delta\in\Delta(\delta(n^*))}\sum_{n'\in\mathcal{N}^\delta}\sum_{c\in\mathcal{C}} p_{n',c}\frac{\eta^u_{n',c}\lambda_{n'}}{\mu_{n'}}$, $\zeta_\Delta(\boldsymbol{\eta}^u) = \sum_{\delta\in\Delta}\sum_{n'\in\mathcal{N}^\delta}\sum_{c\in\mathcal{C}} p_{n',c}\frac{\eta^u_{n',c}\lambda_{n'}}{\mu_{n'}}$, and $\Upsilon(\boldsymbol{\eta}^u) = \frac{1}{2}\sum_{\delta\in\Delta}\sum_{n'\in\mathcal{N}^\delta}\sum_{c\in\mathcal{C}} p_{n',c}\eta^u_{n',c}\boldsymbol{\lambda}_{n'}\nu_{n'}$. This formula indicates that prioritized offloading systems yield greater advantages for users with high real-time requirements. (For more proof details, please refer to Appendix N-A of technical report [52].)

b) The lowest priority users: Similarly, the PAoI of user n_*, who is set to have the lowest priority ($\mathbb{E}[A^p_{n_*}]$), has a worse PAoI than the priority-free case ($\mathbb{E}[A_{n_*}]$). Thus, we gain

$$\begin{aligned}&\mathbb{E}[A_{n_*}] - \mathbb{E}[A^p_{n_*}]\\&= \frac{\Upsilon(\boldsymbol{\eta}^u)}{1-\zeta_\Delta(\boldsymbol{\eta}^u)}(1-\frac{1}{1-\zeta_{\delta(n)-1}(\boldsymbol{\eta}^u)})\\&\leq 0.\end{aligned} \tag{53}$$

This formula demonstrates that the potential drawback of a priority offloading system is the insufficient guarantee of information freshness for users with lower priority. (For more proof details, please refer to Appendix N-B of technical report [52].)

c) The higher priority users: Furthermore, multiple users are assumed to have the same priority level but are instead assigned priorities of δ^0 and $\delta^0 - \hat{\delta}$ in the multi-priority scenario, the resulting difference in freshness can be substantial and unfair:

$$\begin{aligned}&\mathbb{E}[A^p_n|\delta(n)=\delta^0] - \mathbb{E}[A^p_n|\delta(n)=\delta^0-\hat{\delta}]\\&\geq \frac{\Upsilon(\boldsymbol{\eta}^u)(\zeta_{\delta^0}(\boldsymbol{\eta}^u) - \zeta_{\delta^0-\hat{\delta}-1}(\boldsymbol{\eta}^u))}{(1-\zeta_{\delta^0}(\boldsymbol{\eta}^u))(1-\zeta_{\delta^0-1}(\boldsymbol{\eta}^u))(1-\zeta_{\delta^0-\hat{\delta}-1}(\boldsymbol{\eta}^u))}\\&\geq 0.\end{aligned} \tag{54}$$

This formula reveals that the most straightforward and impact approach to getting fresher information is elevating the user's priority; otherwise, its priority would be lowered. (For more proof details, please refer to Appendix N-C of technical report [52].) Based on the conclusions drawn, we have:

$$\begin{aligned}&\mathbb{E}[A^p_n|\delta(n)=\delta^h] - \mathbb{E}[A^p_n|\delta(n)=\delta^l]\\&= \mathbb{E}[A^p_n|\delta(n)=\delta^l] - \mathbb{E}[A^p_n|\delta(n)=\delta^l-\hat{\delta})]\\&\leq 0,\end{aligned} \tag{55}$$

where $\hat{\delta} = \delta^l - \delta^h > 0$ represents the differential priority level, with δ^h and δ^l denoting high and low priorities, respectively. This formula reveals that high-priority users are guaranteed to have lower PAoI values compared to low-priority users.

This subsection highlights that user n can get superior performance as compared to priority-free by allocating it a high priority. Furthermore, allocating user n to a higher level also enhances its performance and allows it to obtain more up-to-date information at the expense of users with lower priority. We substantiate these claims with comprehensive simulations in the next section.

VI. Simulation

In this section, we evaluate our proposed algorithm by answering the following questions:

1) What is the overall utility of our method?
2) Can it schedule multi-priority tasks effectively?
3) Can it deal with heterogeneous and unreliable channels effectively?

A. Experimental Setup

Parameters. In this section, we carry out simulations to evaluate the effectiveness, performance, and computational efficiency of our proposed method. In the priority-free case, we consider 10 servers and 200 users within their respective coverage areas. For the multi-priority case, we examine at least three priorities, allocating users to different priority levels. Moreover, we model the transmission success probability of the channel as a Gaussian distribution $N(0.5, 1)$, following [60]. We set the number of available channels C to be 30, with a bandwidth of $B = 5MHz$, and channel gains were set to unity as in [61]. To account for the heterogeneity of users and servers, the service time of tasks followed a general distribution, where the mean and variance of the distribution are determined by the types of users and servers. Specifically, we set the value of the mean and variance of the general distribution to follow the uniform distribution $U(1, 5)$ and $U(1, 25)$, respectively.

Performance metrics. In the following experiments, we mainly use PAoI and throughput as performance metrics. A lower PAoI signifies fresher information for the user and a reduced number of outdated tasks. Conversely, a higher throughput indicates better utilization of communication and computation resources within the same experimental setup.

Baselines. We compare our proposed algorithm with existing algorithms in the literature to perform a comprehensive

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II
PARAMETERS IN SIMULATION

Parameter	Value
# users (N)	an integer varying between $[1, 200]$
# channels (C)	an integer varying between $[1, 30]$
# servers (M)	an integer varying between $[1, 10]$
the channel condition	channel bandwidth (B) as 5MHz channel gains (R_c) as 1
the service time	obey the general distribution with the mean μ following $U(1, 5)$, and the variance ν following $U(1, 25)$, respectively
the transmission rates	real numbers (Mbit/slot) varying between $[0, 0.5]$
task generation rates	real numbers (Mbit/slot) following $U(0.5, 1.5)$

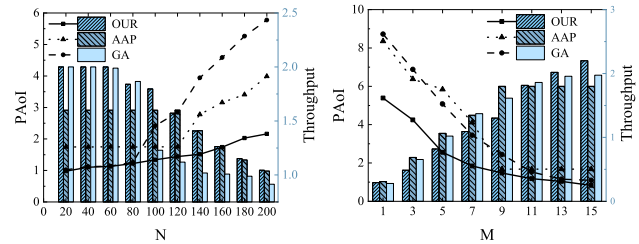
analysis. Specifically, we compare our method with the Age-Aware Policy (AAP) algorithm which utilizes throughput constraints through the Lyapunov optimization method [6]. We also consider the Greedy Control Algorithm (GA), which selects the most reliable channel among the unreliable channels. Additionally, to account for the lack of priority mechanism in AAP and GA, we compare our algorithm with the Priority Scheduling method of Peak Age of Information in Priority Queueing Systems (PAUSE) [43] and the Rate and Age of Information (RAI) method [62]. To ensure a fair comparison, we assume that each user sends the maximum possible number of tasks to the edge server, and the edge server completes the tasks in a First-Come-First-Serve (FCFS) manner.

Implementation. The simulation platform is Matlab R2019a and all the simulations are performed on a laptop with 2.5 GHz Intel Core i7 and 16 GB RAM.

B. Overall Utility

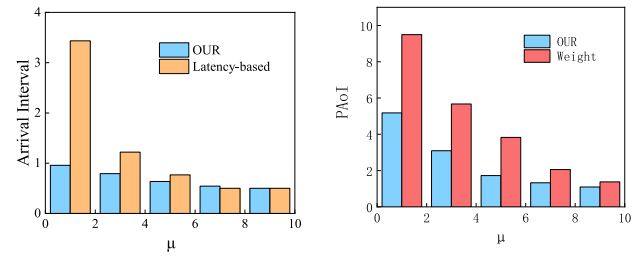
In the assessment of the overall utility, we conducted an analysis of various metrics, such as PAoI and throughput, under different methods. We further compare our PAoI-based approach with the latency-based method and the weight-based method. Moreover, we examined their performance in various settings, including priority allocation and server collaboration.

Firstly, we compare the performance of our proposed algorithm (OUR) with the Greedy Control Algorithm (GA) and the Age-Aware Policy (AAP) algorithm in multi-user and multi-server cases. As depicted in Fig.2(a) and 2(b), the blue bars represent throughput, while the black lines illustrate the Packet Age of Information (PAoI). Our evaluation underscores that the PAoI value and throughput are notably influenced by the number of users (N) and servers (M). Overall, OUR's main strength lies in its consistently superior overall performance in comparison to GA and AAP. Notably, OUR's PAoI excels across diverse parameter settings compared to GA and AAP. However, in scenarios characterized by resource limitations, as illustrated in Fig.2(b), the throughput performance of our algorithm is slightly lower. This is attributed to OUR's predominant emphasis on minimizing PAoI, whereas the AAP method inherently prioritizes throughput with its foundational constraint. Although throughput is not OUR's primary focus, it achieves an optimal or at least suboptimal level when contrasted with the other two algorithms.



(a) PAoI & Throughout vs. N (b) PAoI & Throughout vs. M

Fig. 2. Impact of methods on various metrics.



(a) Arrival Internal vs. μ (b) PAoI vs. μ

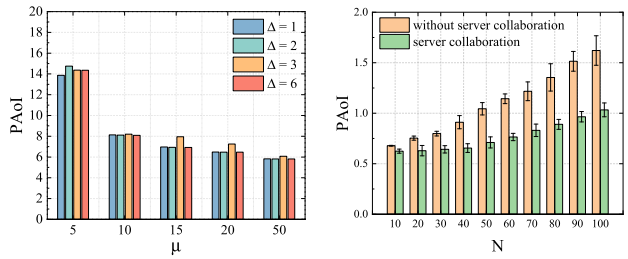
Fig. 3. Performance comparison between different methods.

In our second set of analyses, we compared our approach, which utilizes strict priority control, with latency-based methods and weight-based (flexible priority control). We explore the latency-based method, focusing on the performance metric of arrival interval. As depicted in Fig.3(a), we found that the latency-based method underperforms in resource-constrained scenarios. The high arrival intervals, indicative of reduced frequency of updates, emerged as a notable limitation in latency-based methods. This shortfall makes them less adept for applications like AR/VR, where there is a critical need for swift information updates [5]. We further compared our approach, which employs hard priority control, with the weight-based method that uses flexible priority control. The results, illustrated in Fig.3(b), showed that our approach effectively reduces the Peak Age of Information (PAoI) for high-priority users. This indicates a more efficient delivery of timely information compared to the weight-based method. The advantage of our approach stems from its server-side priority queuing system, which ensures high-priority tasks are processed more promptly. This experiment underlines the effectiveness of our PAoI-based method in scenarios where speed and accuracy of information processing are paramount.

Our third segment of analysis focuses on the impact of the priorities division and server collaboration on the overall PAoI performance. We conducted a simulation for the multi-priority case, as illustrated in Fig.4(a). Here, Δ denotes the degree of priority division, with $\Delta = 1$ indicating no priority distinction and $\Delta = 6$ indicating six priority classes. The simulation results show that the average PAoI of the system is mainly determined by the computing power of the server (μ, task execution time), and the priority division level has little influence on it. Furthermore, we investigate the cases of whether servers are collaborating. Fig.4(b) presents the results for both the server collaboration case and the without-server collaboration case, showing that the PAoI performance of the

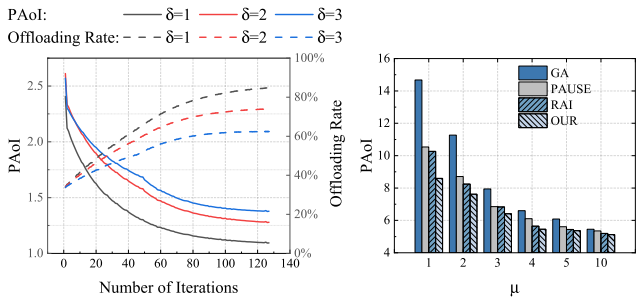
Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



(a) PAoI vs. μ in different priority allocation case (b) PAoI vs. N in the case of whether servers are collaborating

Fig. 4. The performance in various settings.



(a) Performance of users with different priorities (b) Comparison of high priority users in different algorithms

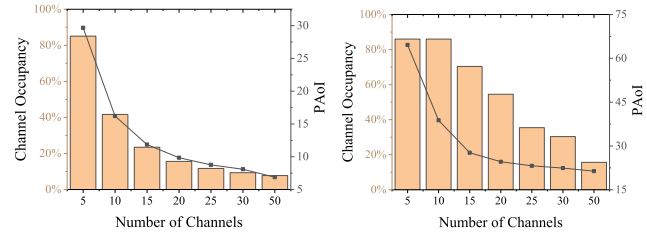
Fig. 5. Performance of priority users.

former outperforms the latter regardless of the number of users. Notably, this feature is more prominent as the number of users N increases. Besides, the PAoI variance in the server collaboration scenario is notably lower, suggesting its stronger system stability.

C. Performance of Priority Tasks

Simulation results show that the proposed algorithm can be extended to multi-class priority scenarios. As shown in Fig.5(a), we set three priority classes (δ lower, higher priority) and then randomly allocated these priority levels to an equal number of users, each of whom had the same quantity of tasks. As the iterations of our algorithm proceed, the average PAoI values of users with various priority levels steadily decrease and eventually tend to stabilize, and the offloading rate gradually increases and eventually tends to be stable. We find that users with higher priority ($\delta = 1$) always get higher offloading rates, which results from more channel resource allocations. In addition, high-priority users' PAoI values are lower since their tasks are scheduled and executed promptly. The above observation implies that high-priority users receive a larger share of channel and computing resources and they are more likely to achieve superior performance.

With three types of priorities (high priority, medium priority, low priority), we compare the proposed multi-class priority method (OUR) with other algorithms (GA, PAUSE, RAI). We pay special attention to the promotion effect of our algorithm for high-priority users, as shown in Fig.5(b). The GA algorithm is one that does not consider priorities, the PAUSE algorithm focuses on the discussion of multiple priorities rather than multiple classes of priorities, and the



(a) High server processing rates (b) Low server processing rates

Fig. 6. PAoI & channel occupancy vs. Number of channels.

RAI algorithm only considers two classes of priorities. The generality of these methods falls short, and they are unable to provide high-priority users with the lower PAoI that they require. However, it is clear that the proposed strategy our approach is always the best one with the lowest PAoI and can be easily implemented across a range of scenarios.

D. Performance and Convergence of Algorithms in Channels

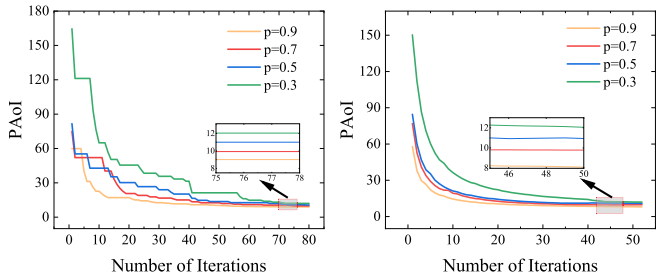
In this subsection, we empirically show the correlation between the algorithm performance and the number of channels, as well as the correlation between the algorithm convergence rate and the channel unreliability.

In order to investigate the potential impact of the number of channels on algorithm performance, we compared the impact of the number of channels on PAoI and average channel occupancy. To this end, we run experiments in two different server processing rates scenarios, as seen in Fig.6(a) and 6(b), respectively. On the one hand, we notice a drop in PAoI as the number climbs, indicating that more tasks may be sent to the server across a more dependable and fast channel as overall channel capacity rises. Particularly, in the scenario of high server processing rates (Fig.6(a)), the server processing efficiency is high, resulting in a lower total PAoI value than in another scenario (Fig.6(b)). On the other hand, as the overall channel capacity increases with the number of channels, the channel occupancy decreases. This ensures that tasks can be transmitted to the server on a more reliable and faster path. However, if the servers' service efficiency becomes lower than the channel transmission efficiency, there is an upper bound to the channel occupancy. This feature is shown in Fig.6(b) as the number of channels gets low and the reason why it does not reach 100% is discussed in detail in Section III.

The convergence rate of distributed algorithms can be hindered by a lack of reliable communication resources. However, both NFPA-NAC and NFPA-ANAC (as shown in Fig.7(a) and 7(b) respectively) are advantageous for the implementation of powerful edge servers. This is due to their ability to efficiently allocate channel and task-scheduling resources. We show the performance of algorithms NFPA-NAC and NFPA-ANAC in different channel conditions ($p = 0.3, 0.5, 0.7, 0.9$) in Fig.7(a) and Fig.7(b) without repeated experiments. We observe that the PAoI values gradually decline and eventually stabilize as the iterations proceed. Moreover, the lower the value of channel condition p, the slower the convergence rate. Additionally, it can be observed that PAoI can be more optimal with better channel conditions

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



(a) NFPA-NAC (b) NFPA-ANAC

Fig. 7. The influence of the unreliable channel on the PAoI.

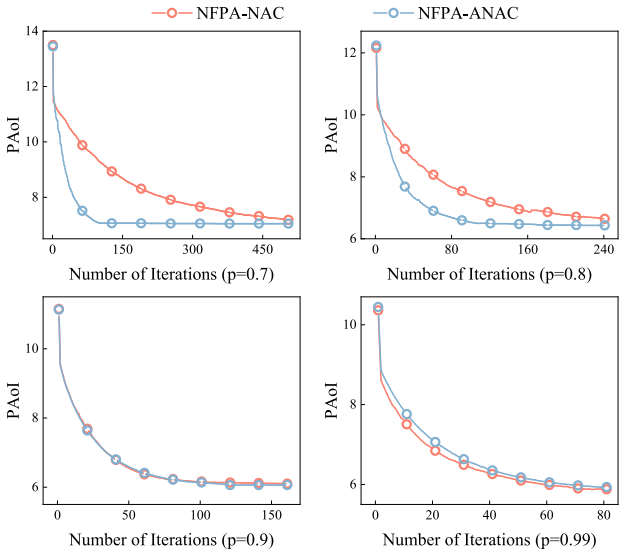


Fig. 8. Comparison with different algorithms.

because users have more options for offloading. We note that in our simulations the algorithm NFPA-NAC cannot successfully iterate in each iteration, while NFPA-ANAC can successfully iterate using the limited information in each iteration.

To illustrate the differences between the synchronous and asynchronous parallel algorithms, specifically NFPA-NAC and NFPA-ANAC, we can refer to Fig.8. This figure depicts the convergence of these two algorithms under various channel conditions ($p = 0.7, 0.8, 0.9, 0.99$). In contrast to Fig.7, we choose a better initial value, more users, and a large number of repeated experiments to help explain the convergence characteristics. Both methods have similar rates of convergence in acceptable channels ($p = 0.9, 0.99$). However, the convergence rate advantage of algorithm NFPA-ANAC, however, is fairly apparent in the ($p = 0.7, 0.8$) channel because of the poorer channel quality. This observation verifies our discussion in Section V-A, as it clearly demonstrates that the asynchronous parallel algorithm (NFPA-ANAC) exhibits greater communication efficiency, particularly in unreliable channel conditions.

VII. Conclusion

In this paper, we proposed a task scheduling method that considered multi-priority users and multi-server collaboration to address the limitations of unreliable channels in current real-time systems and the individual needs of users. We derived the utility function of priority scheduling based on PAoI and designed a set of distributed optimization methods. Specifically, we first considered two simplified problems for the original problem and employed fractional programming as well as ADMM to obtain their solutions. Building upon these solutions and the conclusions drawn therein, we developed an iterative algorithm to solve the original problem. Furthermore, we proposed a distributed asynchronous approach with a sublinear rate of communication defects and discussed the theoretical performance improvement due to the multi-priority mechanism. We implemented the method and conducted extensive simulations to compare it with the existing age-based scheduling strategies. Our results demonstrated the effectiveness and superiority of our method in addressing the requirements of freshness-sensitive users over unreliable channels.

References

[1] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, "A survey on edge computing systems and tools," *Proc. IEEE*, vol. 107, no. 8, pp. 1537–1562, Aug. 2019.

[2] Y. Zhang, J. Ren, J. Liu, C. Xu, H. Guo, and Y. Liu, "A survey on emerging computing paradigms for big data," *Chin. J. Electron.*, vol. 26, no. 1, pp. 1–12, Jan. 2017.

[3] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.

[4] Y. Zou, K. T. Kim, X. Lin, and M. Chiang, "Minimizing age-of-information in heterogeneous multi-channel systems: A new partial-index approach," in *Proc. 22nd Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, Jul. 2021, pp. 11–20.

[5] D. Guo and I.-H. Hou, "Scheduling real-time information-update flows for the optimal confidence in estimation," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1339–1351, May 2021.

[6] J. Sun, L. Wang, Z. Jiang, S. Zhou, and Z. Niu, "Age-optimal scheduling for heterogeneous traffic with timely throughput constraints," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1485–1498, May 2021.

[7] S. Li, C. Li, Y. Huang, B. A. Jalaian, Y. T. Hou, and W. Lou, "Task offloading with uncertain processing cycles," in *Proc. 22nd Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, Jul. 2021, pp. 51–60.

[8] A. M. Bedewy, Y. Sun, R. Singh, and N. B. Shroff, "Optimizing information freshness using low-power status updates via sleep-wake scheduling," in *Proc. 21st Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, Oct. 2020, pp. 51–60.

[9] J. Pan, A. M. Bedewy, Y. Sun, and N. B. Shroff, "Minimizing age of information via scheduling over heterogeneous channels," in *Proc. 22nd Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, Jul. 2021, pp. 111–120.

[10] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1183–1210, May 2021.

[11] Q. Liu, C. Li, Y. T. Hou, W. Lou, and S. Kompella, "Aion: A bandwidth optimized scheduler with AoI guarantee," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2021, pp. 1–10.

[12] C. Li, Q. Liu, S. Li, Y. Chen, Y. T. Hou, and W. Lou, "On scheduling with AoI violation tolerance," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2021, pp. 1–9.

[13] J. Moon and T. Başar, "Minimax control over unreliable communication channels," *Automatica*, vol. 59, pp. 182–193, Sep. 2015.

[14] H. Lv, Z. Zheng, F. Wu, and G. Chen, "Strategy-proof online mechanisms for weighted AoI minimization in edge computing," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1277–1292, May 2021.

[15] R. D. Yates, "Status updates through networks of parallel servers," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2018, pp. 2281–2285.

[16] A. M. Bedewy, Y. Sun, and N. B. Shroff, "Optimizing data freshness, throughput, and delay in multi-server information-update systems," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2016, pp. 2569–2573.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[17] F. Li, Y. Sang, Z. Liu, B. Li, H. Wu, and B. Ji, "Waiting but not aging: Optimizing information freshness under the pull model," *IEEE/ACM Trans. Netw.*, vol. 29, no. 1, pp. 465–478, Feb. 2021.

[18] J. Sun, Z. Jiang, B. Krishnamachari, S. Zhou, and Z. Niu, "Closed-form Whittle's index-enabled random access for timely status update," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1538–1551, Mar. 2020.

[19] B. Sombabu and S. Moharir, "Age-of-information based scheduling for multi-channel systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4439–4448, Jul. 2020.

[20] J. Zhang, X. Zhou, T. Ge, X. Wang, and T. Hwang, "Joint task scheduling and containerizing for efficient edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 8, pp. 2086–2100, Aug. 2021.

[21] S. Halder, A. Ghosal, and M. Conti, "Dynamic super round-based distributed task scheduling for UAV networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 1014–1028, Feb. 2023.

[22] U. Saleem, Y. Liu, S. Jangsher, Y. Li, and T. Jiang, "Mobility-aware joint task scheduling and resource allocation for cooperative mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 360–374, Jan. 2021.

[23] A. A. Al-Habob, O. A. Dobre, A. G. Armada, and S. Muhaidat, "Task scheduling for mobile edge computing using genetic algorithm and conflict graphs," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8805–8819, Aug. 2020.

[24] X. Wang, Z. Ning, and S. Guo, "Multi-agent imitation learning for pervasive edge computing: A decentralized computation offloading algorithm," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 411–425, Feb. 2021.

[25] L. Xujie, T. Jing, X. Yuan, and S. Ying, "Mobility-aware multi-task migration and offloading scheme for Internet of Vehicles," *Chin. J. Electron.*, vol. 32, no. 6, pp. 1192–1202, Nov. 2023.

[26] Y. Chen, J. Hu, J. Zhao, and G. Min, "QoS-aware computation offloading in LEO satellite edge computing for IoT: A game-theoretical approach," *Chin. J. Electron.*, vol. 33, pp. 1–12, Mar. 2023.

[27] Y. Bai, H. Zhao, X. Zhang, Z. Chang, R. Jäntti, and K. Yang, "Toward autonomous multi-UAV wireless network: A survey of reinforcement learning-based approaches," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 3038–3067, Oct. 2023.

[28] Y. Chen, Z. Chang, G. Min, S. Mao, and T. Hämäläinen, "Joint optimization of sensing and computation for status update in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 8230–8243, Nov. 2023.

[29] Y. Chen, J. Zhao, J. Hu, S. Wan, and J. Huang, "Distributed task offloading and resource purchasing in NOMA-enabled mobile edge computing: Hierarchical game theoretical approaches," *ACM Trans. Embedded Comput. Syst.*, early access, May 2023, doi: 10.1145/3597023.

[30] F. Liu, J. Huang, and X. Wang, "Joint task offloading and resource allocation for device-edge-cloud collaboration with subtask dependencies," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3027–3039, Jul./Sep. 2023.

[31] M. A. Abd-Elmagid, H. S. Dhillon, and N. Pappas, "A reinforcement learning framework for optimizing age of information in RF-powered communication systems," *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 4747–4760, Aug. 2020.

[32] Y.-P. Hsu, E. Modiano, and L. Duan, "Scheduling algorithms for minimizing age of information in wireless broadcast networks with random arrivals," *IEEE Trans. Mobile Comput.*, vol. 19, no. 12, pp. 2903–2915, Dec. 2020.

[33] U. Karabulut, A. Awada, I. Viering, M. Simsek, and G. P. Fettweis, "Spatial and temporal channel characteristics of 5G 3D channel model with beamforming for user mobility investigations," *IEEE Commun. Mag.*, vol. 56, no. 12, pp. 38–45, Dec. 2018.

[34] Y. Zeng and R. Zhang, "Optimized training for net energy maximization in multi-antenna wireless energy transfer over frequency-selective channel," *IEEE Trans. Commun.*, vol. 63, no. 6, pp. 2360–2373, Jun. 2015.

[35] A. Kosta, N. Pappas, and V. Angelakis, "Age of information: A new concept, metric, and tool," *Found. Trends Netw.*, vol. 12, no. 3, pp. 162–259, 2017.

[36] P. Zou, O. Ozel, and S. Subramaniam, "Optimizing information freshness through computation—Transmission tradeoff and queue management in edge computing," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 949–963, Apr. 2021.

[37] M. A. Abd-Elmagid and H. S. Dhillon, "Age of information in multi-source updating systems powered by energy harvesting," *IEEE J. Sel. Areas Inf. Theory*, vol. 3, no. 1, pp. 98–112, Mar. 2022.

[38] L. Huang and E. Modiano, "Optimizing age-of-information in a multi-class queueing system," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2015, pp. 1681–1685.

[39] Z. Liu, L. Huang, B. Li, and B. Ji, "Anti-aging scheduling in single-server queues: A systematic and comparative study," *J. Commun. Netw.*, vol. 23, no. 2, pp. 91–105, Apr. 2021.

[40] A. Maatouk, Y. Sun, A. Ephremides, and M. Assaad, "Status updates with priorities: Lexicographic optimality," in *Proc. 18th Int. Symp. Modelling Optim. Mobile, Ad Hoc, Wireless Netw. (WiOPT)*, Jun. 2020, pp. 1–8.

[41] A. Maatouk, M. Assaad, and A. Ephremides, "Age of information with prioritized streams: When to buffer preempted packets?" in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2019, pp. 325–329.

[42] S. K. Kaul and R. D. Yates, "Age of information: Updates with priority," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2018, pp. 2644–2648.

[43] J. Xu and N. Gautam, "Peak age of information in priority queuing systems," *IEEE Trans. Inf. Theory*, vol. 67, no. 1, pp. 373–390, Jan. 2021.

[44] S. Yue et al., "TODG: Distributed task offloading with delay guarantees for edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 7, pp. 1650–1665, Jul. 2022.

[45] Q. Wang and S. Chen, "Latency-minimum offloading decision and resource allocation for fog-enabled Internet of Things networks," *Trans. Emerg. Telecommun. Technol.*, vol. 31, no. 12, p. e3880, Dec. 2020.

[46] J. Ren et al., "An efficient two-layer task offloading scheme for MEC system with multiple services providers," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2022, pp. 1519–1528.

[47] W. R. Ghanem, V. Jamali, Y. Sun, and R. Schober, "Resource allocation for multi-user downlink MISO OFDMA-URLLC systems," *IEEE Trans. Commun.*, vol. 68, no. 11, pp. 7184–7200, Nov. 2020.

[48] F. Callegati, M. Casoni, and C. Raffaelli, "Packet optical networks for high-speed TCP-IP backbones," *IEEE Commun. Mag.*, vol. 37, no. 1, pp. 124–129, Jan. 1999.

[49] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 4th Quart., 2017.

[50] X. Zhou, C. Wang, R. Tang, and M. Zhang, "Channel estimation based on statistical frames and confidence level in OFDM systems," *Appl. Sci.*, vol. 8, no. 9, p. 1607, Sep. 2018.

[51] V. Patil and H. Kulkarni, "Comparison of confidence intervals for the Poisson mean: Some new aspects," *REVSTAT Stat. J.*, vol. 10, no. 2, pp. 211–227, 2012.

[52] N. Qiao, S. Yue, Y. Zhang, and J. Ren, "PoPeC: PAoI-centric task offloading with priority over unreliable channels," 2023, *arXiv:2303.15119*.

[53] S.-H. Kim and W. Whitt, "Statistical analysis with Little's Law," *Oper. Res.*, vol. 61, no. 4, pp. 1030–1045, 2013.

[54] Q. He, D. Yuan, and A. Ephremides, "On optimal link scheduling with min–max peak age of information in wireless systems," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2016, pp. 1–7.

[55] J. Du, L. Zhao, J. Feng, and X. Chu, "Computation offloading and resource allocation in mixed fog/cloud computing systems with min–max fairness guarantee," *IEEE Trans. Commun.*, vol. 66, no. 4, pp. 1594–1608, Apr. 2018.

[56] S. Boyd, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2010.

[57] W. Dinkelbach, "On nonlinear fractional programming," *Manage. Sci.*, vol. 13, no. 7, pp. 492–498, Mar. 1967.

[58] M. Hong, Z.-Q. Luo, and M. Razaviyayn, "Convergence analysis of alternating direction method of multipliers for a family of nonconvex problems," *SIAM J. Optim.*, vol. 26, no. 1, pp. 337–364, Jan. 2016.

[59] M. Hong, "A distributed, asynchronous, and incremental algorithm for nonconvex optimization: An ADMM approach," *IEEE Trans. Control Netw. Syst.*, vol. 5, no. 3, pp. 935–945, Sep. 2018.

[60] G. Scutari, D. P. Palomar, and S. Barbarossa, "Asynchronous iterative water-filling for Gaussian frequency-selective interference channels," *IEEE Trans. Inf. Theory*, vol. 54, no. 7, pp. 2868–2878, Jul. 2008.

[61] N. Balasubramanian, A. Balasubramanian, and A. Venkataramani, "Energy consumption in mobile phones: A measurement study and implications for network applications," in *Proc. 9th ACM SIGCOMM Conf. Internet Meas.*, Nov. 2009, pp. 280–293.

[62] M. P. Abdollahi, H. Azarhava, A. Haghrah, and J. M. Niya, "On the rate and age of information for non-preemptive systems with prioritized arrivals and deterministic packet deadlines in IoT networks," *Ad Hoc Netw.*, vol. 124, Jan. 2022, Art. no. 102717.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Nan Qiao (Graduate Student Member, IEEE) received the B.Sc. degree in computer science from Central South University, China, where he is currently pursuing the Ph.D. degree in computer science. His research interests include wireless communications, distributed optimization, and the Internet of Things.

Sheng Yue (Member, IEEE) received the B.Sc. degree in mathematics and the Ph.D. degree in computer science from Central South University, China, in 2017 and 2022, respectively. Currently, he is a Post-Doctoral Researcher with the Department of Computer Science and Technology, Tsinghua University, China. His research interests include network optimization, distributed learning, and reinforcement learning.

Yongmin Zhang (Senior Member, IEEE) received the Ph.D. degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2015. From 2015 to 2019, he was a Post-Doctoral Research Fellow with the Department of Electrical and Computer Engineering, University of Victoria, BC, Canada. He is currently a Professor with the School of Computer Science and Engineering, Central South University, Changsha, China. His research interests include the IoTs, smart grids, and mobile computing. He won the Best Paper Award of IEEE PIMRC 2012 and the IEEE Asia–Pacific (AP) Outstanding Paper Award in 2018.

Ju Ren (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in computer science from Central South University, China, in 2009, 2012, and 2016, respectively. Currently, he is an Associate Professor with the Department of Computer Science and Technology, Tsinghua University, China. His research interests include the Internet of Things, edge computing, edge intelligence, and security and privacy. He received several Best Paper Awards from IEEE flagship conferences, including IEEE ICC'19 and IEEE HPCC'19, the IEEE TCSC Early Career Researcher Award in 2019, and the IEEE ComSoc Asia–Pacific Best Young Researcher Award in 2021. He was recognized as a highly cited researcher by Clarivate (2020–2022). He has served as the General Co-Chair for IEEE BigDataSE'20; the TPC Co-Chair for IEEE BigDataSE'19; the Track Co-Chair for IEEE ICDCS'24; the Poster Co-Chair for IEEE MASS'18; and the Symposium Co-Chair for IEEE/CIC ICCC'23 and 2019, I-SPAN'18, and IEEE VTC'17 Fall. He serves as an Associate Editor for many journals, including IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON CLOUD COMPUTING, and IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 05,2024 at 09:50:28 UTC from IEEE Xplore. Restrictions apply.